

Spark avec Google Colab

Objectifs : Découvrir et manipuler Spark avec PySpark :

- Comprendre **RDD**
- Créer et transformer des **DataFrames**
- Faire des requêtes SQL sur des **DataFrames**

PySpark : C'est une bibliothèque qui permet d'utiliser toutes les fonctionnalités de Spark (RDD, DataFrame, SQL, MLlib...) directement en Python, au lieu d'écrire du code en Scala ou Java.

Google Colab (Colaboratory) est un environnement gratuit dans le cloud pour exécuter du code Python dans le navigateur. Il permet d'installer et d'utiliser **PySpark**, donc d'exécuter **Spark** complètement depuis le navigateur,

1. Ouvrir Google Colab

- Ouvrir le lien sur <https://colab.research.google.com>
- Cliquer sur “**Nouveau notebook**”
- Taper ce code

```
!apt-get install openjdk-8-jdk-headless -qq > /dev/null

!wget -q http://archive.apache.org/dist/spark/spark-3.4.0/spark-3.4.0-bin-hadoop3.tgz

!tar xf spark-3.4.0-bin-hadoop3.tgz

!pip install -q pyspark
```

- Exécuter la cellule
- Colab va installer Java, télécharger Spark et installer PySpark

Pour verifier les installations

```
!java -version

import pyspark

print(pyspark.__version__)
```

2. Configurer Spark

Après l'installation, ajouter une nouvelle cellule avec ce code pour configurer Spark :

```
import os

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
os.environ["SPARK_HOME"] = "/content/spark-3.4.0-bin-hadoop3"

from pyspark.sql import SparkSession

spark =
SparkSession.builder.master("local[*]").appName("SparkColab").getOrCreate()
```

3. Manipulation des RDDs

Exemple 1 RDD simple : Calcul simple sur des nombres

```
# Création d'un RDD

rdd = spark.sparkContext.parallelize([1,2,3,4,5])

# spark.sparkContext :le moteur Spark

# parallelize(data) : Spark coupe la liste en partitions pour pouvoir les traiter
en parallèle.

# Transformation : doubler chaque valeur

rdd2 = rdd.map(lambda x: x * 2)

# Spark applique la fonction lambda x: x*2 à chaque élément du RDD.

# Mais Spark n'exécute rien tant qu'il n'y a pas une action.

# Action : afficher les résultats

# collect() est une action : elle demande à Spark d'exécuter réellement le calcul.
print(rdd2.collect())
```

L'affichage de ce code est

```
[2, 4, 6, 8, 10]
```

Exemple 2 RDD simple : Transformation en majuscule

```
from pyspark.sql import SparkSession

# Créer SparkSession

spark =
SparkSession.builder.master("local[*]").appName("SimpleTextMapRDD").getOrCreate()

# Exemple de texte

text = ["spark est génial", "pyspark est puissant", "map et reduce"]

# Créer RDD à partir de la liste de phrases

rdd = spark.sparkContext.parallelize(text)

# Transformation : convertir chaque phrase en majuscules

upper_rdd = rdd.map(lambda line: line.upper())

# Action : afficher les résultats

print(upper_rdd.collect())
```

4. Manipulation des DataFrames**Exemple 1 DataFrame simple**

```
from pyspark.sql import SparkSession

# Créer la SparkSession

spark =
SparkSession.builder.master("local[*]").appName("SparkSQLExample").getOrCreate()

# Exemple de données

data = [
    {"nom": "Ahmed", "age": 28, "salaire": 3000},
    {"nom": "Fatima", "age": 32, "salaire": 3500},
    {"nom": "Omar", "age": 25, "salaire": 2800},
    {"nom": "Layla", "age": 30, "salaire": 3200}
]

# Créer le DataFrame

df = spark.createDataFrame(data)

# Afficher le DataFrame

df.show()
```

Affichage

age	nom	salaire
28	Ahmed	3000
32	Fatima	3500
25	Omar	2800
30	Layla	3200

Remarque. pour créer la table temporaire pour SQL

```
df.createOrReplaceTempView("employees")
```

Requete 1 Sélectionner les personnes avec salaire > 3000

```
#avec DataFrame
df.filter(df.salaire > 3000).show()
df.filter(df.salaire >= 3000).count()

#avec Spark SQL
result = spark.sql("SELECT nom, age, salaire FROM employees WHERE salaire > 3000")
result.show()
```

Requete 2 Trier par age croissant

```
# avec DataFrame
df.orderBy(df.age.asc()).show()

#avec Spark SQL
spark.sql("SELECT * FROM employees ORDER BY age ASC").show()
```

Requete 3 Calculer le salaire moyen

```
# avec DataFrame
df.agg({"salaire": "avg"}).show()

#avec Spark SQL
spark.sql("SELECT * FROM employees ORDER BY age ASC").show()
```

Requete 4 Afficher par ordre croissant des employes dont le salaire est superieur a 3000

```
# avec DataFrame
df.filter(df.salaire > 3000).orderBy(df.age.asc()).select("nom", "salaire").show()

# avec Spark SQL

spark.sql("SELECT nom,salaire FROM employes WHERE salaire >3000 ORDER BY age
ASC").show()
```

Exemple 2 DataFrame Word Count

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col, count

spark =
SparkSession.builder.master("local[*]").appName("WordCountDF").getOrCreate()

text = ["hello world", "hello spark", "hello colab", "spark is fun"]

# Créer DataFrame

df = spark.createDataFrame([(line,) for line in text], ["line"])

# Split en mots et compter

word_counts = df.select(explode(split(col("line"), " ")).alias("word")) \
\
                .groupBy("word") \
                .agg(count("word").alias("count"))

word_counts.show()
```

L'affichage est le suivant

```
+-----+-----+
| word | count |
+-----+-----+
|hello|      3|
|spark|      2|
|world|      1|
|  fun|      1|
|   is|      1|
|colab|      1|
+-----+-----+
```

Exemple 3 DataFrame MapReduce simple**Afficher le total des ventes par vendeur****Etape 1 : Creation du DataFrame**

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum

# Créer la session Spark

spark =
SparkSession.builder.master("local[*]").appName("ExempleVentes").getOrCreate()

# Exemple de DataFrame : ventes
data = [
    {"vendeur": "Ahmed", "produit": "A", "quantité": 5, "prix_unitaire": 100},
    {"vendeur": "Fatima", "produit": "B", "quantité": 3, "prix_unitaire": 200},
    {"vendeur": "Omar", "produit": "A", "quantité": 2, "prix_unitaire": 100},
    {"vendeur": "Layla", "produit": "C", "quantité": 7, "prix_unitaire": 150},
    {"vendeur": "Ahmed", "produit": "B", "quantité": 1, "prix_unitaire": 200},
]

#df = spark.createDataFrame(data) ou pour affichage plus correct
df = df.select("vendeur", "produit", "quantité", "prix_unitaire")
df.show()

```

Affichage

prix_unitaire	produit	quantité	vendeur
100	A	5	Ahmed
200	B	3	Fatima
100	A	2	Omar
150	C	7	Layla
200	B	1	Ahmed

Etape 2 : Map Calculer le montant de chaque vente

```
df_mapped = df.withColumn("montant", col("quantité") * col("prix_unitaire"))
df_mapped.show()
```

Affichage

prix_unitaire	produit	quantité	vendeur	montant
100	A	5	Ahmed	500
200	B	3	Fatima	600
100	A	2	Omar	200
150	C	7	Layla	1050
200	B	1	Ahmed	200

Etape 3 : Garder seulement les ventes > à 500

```
df_filtered = df_mapped.filter(col("montant") > 500)
df_filtered.show()
```

Affichage

prix_unitaire	produit	quantité	vendeur	montant
200	B	3	Fatima	600
150	C	7	Layla	1050

Etape 4 : Reduce Aggregation total des ventes par vendeur

```
df_grouped =
df_mapped.groupBy("vendeur").agg(sum("montant").alias("total_ventes"))
df_grouped.show()
```

Affichage

+-----+-----+	
vendeur	total_ventes
+-----+-----+	
Fatima	600
Ahmed	700
Omar	200
Layla	1050
+-----+-----+	
